

RBDQ: A Reliable LLM-based Text-to-SQL System for Business Data Queries

Fenglin Bi
School of Data Science and
Engineering, East China Normal
University
Shanghai, China
fenglin@stu.ecnu.edu.cn

Dongdong Cao
ByteDance Inc
Shanghai, China
caodongdong.064@bytedance.com

Zhiyu Wang
ByteDance Inc
Shanghai, China
wangzhiyu.derek@bytedance.com

Yang Chen
Shanghai Key Lab of Intelligent
Information Processing, School of
Computer Science, Fudan University
Shanghai, China
chenyang@fudan.edu.cn

Fangliang Zhao
ByteDance Inc
Shanghai, China
fangliang.zhao@bytedance.com

Tao Hu
ByteDance Inc
Shanghai, China
jacky.z@bytedance.com

Zhi Li
ByteDance Inc
Shanghai, China
zhi.li@bytedance.com

Yanbin Zhang*
School of Data Science and
Engineering, East China Normal
University
Shanghai, China
ybzhang@dase.ecnu.edu.cn

Wei Wang*
School of Data Science and
Engineering, East China Normal
University
Shanghai, China
wwang@dase.ecnu.edu.cn

Abstract

Using large language models (LLMs) to convert natural language (NL) into SQL simplifies data access for users by allowing them to use everyday language. However, business departments often distrust LLM-based text-to-SQL systems due to the probabilistic nature of SQL generation, which can result in incorrect but executable SQL queries caused by model hallucinations. This leads to significant concerns regarding the accuracy and reliability of the queried data. In this paper, we present RBDQ, a novel LLM-based text-to-SQL system designed to address the unique challenges of business data queries. RBDQ innovatively introduces the Hierarchical Metrics Query Method and integrates advanced Retrieval-Augmented Generation (RAG) methods along with a self-reflection mechanism to tackle these challenges. RBDQ effectively meets the requirements of business metric queries in real-world scenarios. Currently implemented in the Quality Assurance department at ByteDance, RBDQ has significantly improved operational efficiency and query flexibility. Our experiments demonstrate the system's effectiveness, achieving an Execution Accuracy of 96.20%.

*Co-corresponding authors who contributed equally to this work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW Companion '25, April 28-May 2, 2025, Sydney, NSW, Australia
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1331-6/2025/04
<https://doi.org/10.1145/3701716.3715257>

CCS Concepts

• **Information systems** → **Information retrieval**; • **Computing methodologies** → *Natural language processing*.

Keywords

Text-to-SQL, Large Language Models, Retrieval-Augmented Generation, Business Data Queries

ACM Reference Format:

Fenglin Bi, Dongdong Cao, Zhiyu Wang, Yang Chen, Fangliang Zhao, Tao Hu, Zhi Li, Yanbin Zhang, and Wei Wang. 2025. RBDQ: A Reliable LLM-based Text-to-SQL System for Business Data Queries. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, April 28-May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3701716.3715257>

1 Introduction

Text-to-SQL converts (translates) natural language questions into Structured Query Language (SQL) queries that databases can execute. This approach makes it easier for users to access data by using everyday language. It is used in many fields, including healthcare, finance, and logistics. As businesses rely more on data-driven decisions, being able to get insights from databases without knowing SQL is essential to remain competitive.

Text-to-SQL systems are becoming increasingly important as more organizations rely on relational databases to handle ever-growing amounts of data across various sectors. According to the 2023 Stack Overflow Developer Survey, 51.52% of professional developers report using SQL in their work, making it one of the most widely used programming languages today. However, SQL often poses a barrier for non-technical users due to its specialized syntax

and technical nature. Consequently, text-to-SQL systems bridge the gap between the vast data stores housed in relational databases and the ability of many users to retrieve and analyze those data efficiently [7].

Large language models (LLMs), such as the GPT series [1, 2, 16], have gained significant attention in recent years due to their capability to generate coherent and fluent text. Researchers have begun exploring the potential of LLMs for text-to-SQL by leveraging their extensive knowledge reserves and superior generation capabilities.

However, LLM-based text-to-SQL systems face several inherent technical challenges, including linguistic complexity and ambiguity [4, 17], schema understanding and representation [27], rare and complex SQL operations, and cross-domain generalization [26]. In the context of business data queries, there are additional specific challenges: 1) **Concise and Incomplete Queries**: User queries are generally concise and omit many known details (e.g., online bugs in CD), as illustrated in Figure 1a. 2) **Model Hallucinations**: Even when relevant user knowledge and contextual information are supplemented, the hallucination problem can still result in the large model omitting the retrieved information during output, as shown in Figure 1b. 3) **Inconsistent SQL Outputs**: The same query might produce different SQL statements. Some of these statements are correct but increase unnecessary caching and performance overhead, while others are incorrect and non-executable, as depicted in Figure 1c. 4) **Reliability and Traceability**: The results must be reliable and traceable to ensure that business departments can trust the query outcomes.

To address these challenges, we propose RBDQ (Reliable Business Data Queries System), an LLM-based text-to-SQL system specifically designed to tackle the unique challenges of business data queries. We introduce an innovative Hierarchical Metrics Query Method that ensures the reliability and accuracy of query results. Additionally, our approach integrates state-of-the-art Retrieval-Augmented Generation (RAG) methods [13] and a self-reflection mechanism to enhance problem-solving performance in LLMs [18]. By combining best practices in prompt engineering, chain-of-thought (CoT) prompting, question representation, and selecting few-shot examples, RBDQ effectively meets the requirements of business metric queries in real-world scenarios. Currently, RBDQ is being utilized in the Quality Assurance department at ByteDance for daily metric query tasks, demonstrating its practical applicability and effectiveness by enhancing operational efficiency by approximately 5%.

The key contributions of this paper are as follows:

- (1) We propose a novel Hierarchical Metrics Query Method for business metrics, addressing the issue of distrust in LLMs by business departments.
- (2) We introduce the RBDQ architecture, presenting a comprehensive solution for business data queries, effectively handling concise and incomplete user queries.
- (3) We created a business-specific dataset and demonstrated the effectiveness of RBDQ through experiments.

These contributions collectively advance the field of text-to-SQL systems, particularly in the context of business data queries, by providing a robust, accurate, and reliable solution.

2 Related Work

2.1 Evolution of the Text-to-SQL Technology

Early text-to-SQL systems relied on rule-based approaches, using manually crafted grammar rules and heuristics to translate natural language queries into SQL commands. Although effective for simple and domain-specific databases, these methods struggled with complex queries and various schemas, as seen in systems such as LUNAR and NaLIX [6, 9]. The advent of deep learning around 2017 marked a significant shift, with models like Seq2SQL and SQLNet leveraging sequence-to-sequence architectures to improve performance and scalability [10, 29]. Transformer-based models such as BERT and TaBERT further improved understanding of database schemas and user intent, improving generalization to unseen databases [11, 14]. Pre-trained Language Models (PLMs) like BERT and GPT introduced major improvements by capturing richer semantic relationships between user queries and database schemas, although they required significant task-specific fine-tuning [2, 14]. More recently, Large Language Models (LLMs) such as GPT-3 have reduced the need for extensive fine-tuning, offering greater flexibility and efficiency in handling complex, cross-domain queries through prompt engineering [23]. These advancements collectively highlight the evolution from rule-based systems to sophisticated neural network-based models in the text-to-SQL domain.

In the context of text-to-SQL, LLMs capture more complex relationships between natural language queries and structured database schemas. Unlike pre-trained language models (PLMs), which require extensive task-specific fine-tuning and schema linking, LLMs can handle zero-shot and few-shot scenarios more effectively due to their large-scale pretraining and advanced reasoning capabilities [28]. For instance, studies have demonstrated that models like Codex achieve high performance in generating SQL queries with minimal prompt engineering [21]. Despite these advancements, challenges such as handling ambiguous queries and optimizing SQL statements for performance and correctness persist [19].

In particular, the context of Business Data Queries demands high reliability and traceability of the retrieved data. The inherent probabilistic nature of LLMs can lead to issues such as hallucinations [8], where the model generates incorrect or misleading information, thus compromising the trustworthiness of the query results. This is especially critical in business scenarios where data reliability and accountability are paramount. Addressing these challenges requires further research into enhancing the robustness and reliability of LLM-based text-to-SQL systems to meet the stringent requirements of business data analysis.

2.2 LLM-based Text-to-SQL System

Gao et al. [5] systematically explore LLM-based methods for text-to-SQL and combine the best techniques to create DAIL-SQL. They consider five options for formatting the database schema and user questions, four options for selecting the most relevant few-shot examples, and two options for formatting these examples. By synthesizing best practices, they introduce sophisticated new methods—DAIL Selection and DAIL Organization—for selecting and formatting few-shot examples. We draw inspiration from this work, particularly in the area of question representation and selecting few-shot examples.

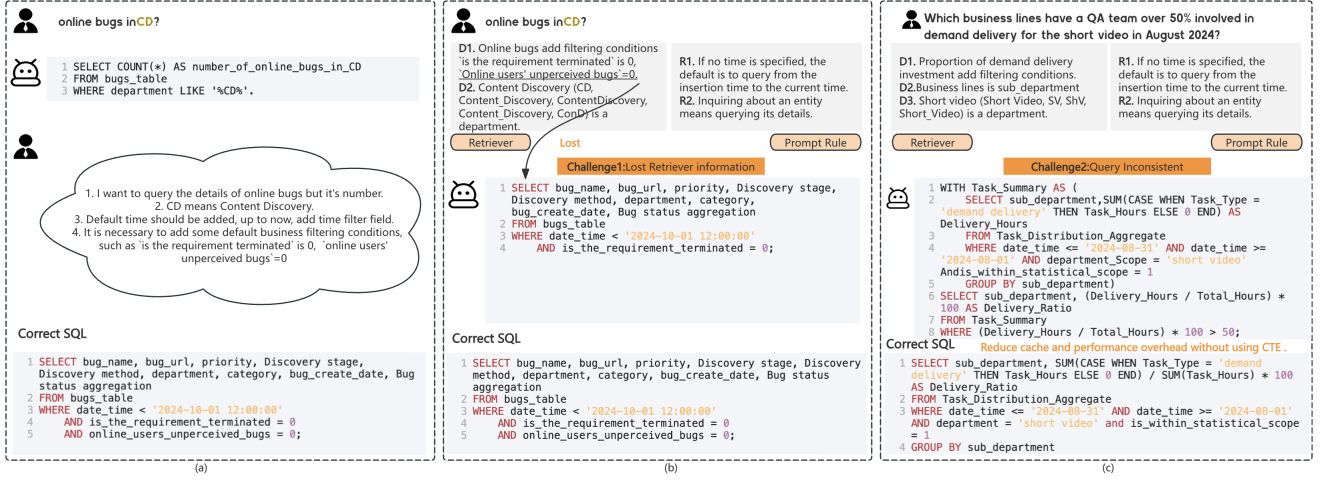


Figure 1: Challenges in Business Data Query

Pourreza and Rafiei [15] propose DIN-SQL, which decomposes questions into smaller sub-questions. Methods such as chain-of-thought prompting [25] and least-to-most prompting [30] have been used to improve LLM performance on tasks that can be broken down into multiple steps, such as mathematical word problems and compositional generalization [3, 12]. However, their approach does not address issues like model hallucinations, which can result in queries that appear to run correctly but do not return the intended results, thus failing to ensure the reliability required in business scenarios.

Thorpe et al. [20] propose Dubo-SQL, which utilizes a SQL compiler to execute SQL queries generated by LLMs. If the compiler encounters an error, the SQL query and the corresponding error message are sent back to the LLM for correction. Similarly, MAC-SQL [22] employs a tool referred to as a "refiner" to correct SQL syntax errors. While this approach resembles a self-reflection mechanism, it focuses on ensuring the correct execution of SQL queries. However, addressing logical errors remains a significant challenge in the text-to-SQL domain.

3 Methodology

In this section, we describe the hierarchical metric query method and the RBDQ system.

3.1 Hierarchical Metrics Query Method

The *Hierarchical Metrics Query Method* consists of two main components: hierarchical metric knowledge construction and hierarchical metric chain of thought.

3.1.1 Hierarchical Metric Knowledge Construction. In the context of business metric queries, the variety and complexity of the metrics and their associated terminology can be substantial. To standardize the expression and calculation logic of these metrics, we have abstracted a Hierarchical Metric Construction method. This method

involves the abstraction of sub-metrics, composite metrics, and calculation logic.

Sub-metrics represent the most granular level of metric calculation logic, such as the number of bugs, bug details, and the number of online bugs. We have constructed SQL templates for these sub-metrics, incorporating both necessary and optional parameters to ensure the reliability of each sub-metric query. Composite metrics are derived from a series of logical operations on sub-metrics. For example, the online bug rate is calculated by dividing the number of online bugs by the total number of bugs. We have abstracted various calculation logics, such as rate calculation, average value calculation, and ranking.

This method allows us to standardize different SQL query syntaxes into a unified SQL template, ensuring consistency in queries. By employing the hierarchical metric chain of thought (Section 3.1.2), we invoke sub-metric SQL templates and use a parameter validation mechanism to further ensure the completeness of user query information.

Through *Hierarchical Metrics Query Method*, we combine the advantages of code validation (SQL) and large model planning (metric calculation logic), balancing the reliability and robustness requirements of metric queries.

3.1.2 Hierarchical Metric Chain of Thought. Building on the knowledge of sub-metrics, composite metrics, and calculation logic, we have developed a *Hierarchical Metric Chain of Thought* to guide the *Planning Steps* of large model outputs. This approach leverages our constructed sub-metrics, composite metrics, and calculation logic to accurately query the results.

As illustrated in Figure 2, the conventional text-to-SQL chain of thought includes understanding the natural language query, finding relevant tables, columns, and cell values, and writing SQL based on the understanding of the natural language and database. In contrast, the Hierarchical Metric Chain of Thought involves several additional steps to ensure accuracy and reliability:

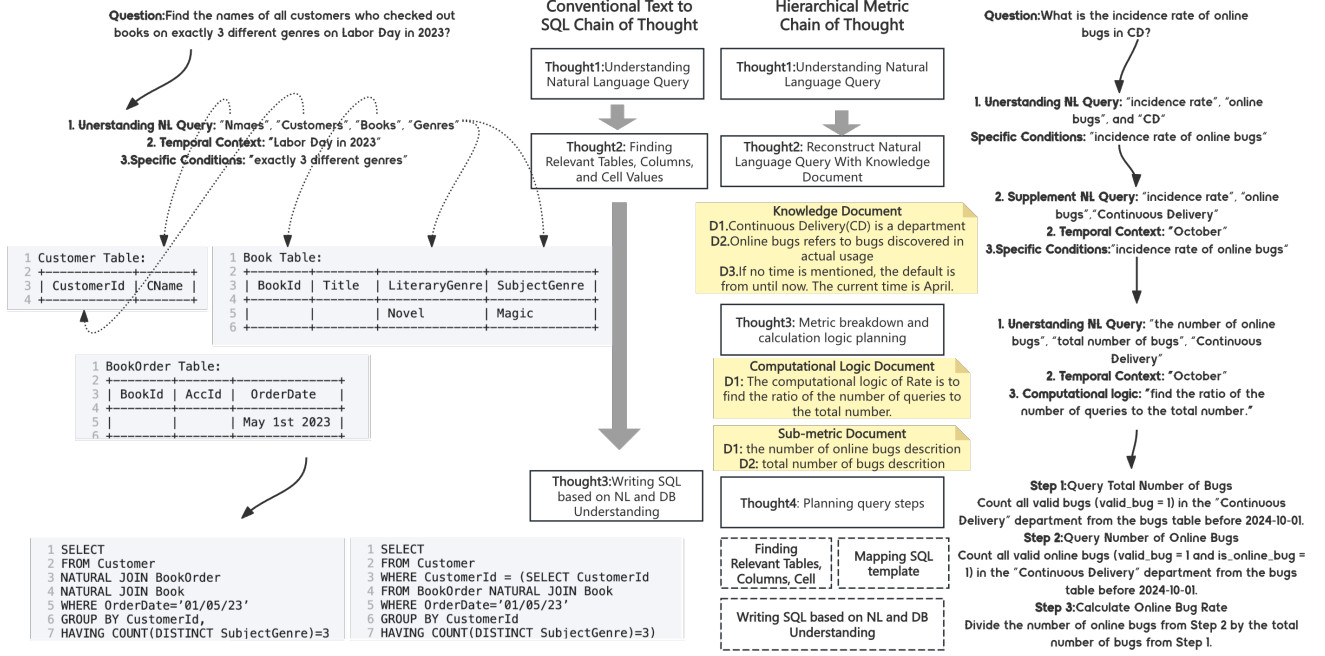


Figure 2: Hierarchical Metric Chain of Thought

1. Understanding Natural Language Query: Given the NL query "What is the incidence rate of online bugs in CD systems?", the first task is to grasp the user's intent and identify key components. Key elements include: 1) Entities or Attributes: "incidence rate", "online bugs", and "CD systems"; 2) Specific Conditions: "incidence rate of online bugs". The goal is to understand that the user wants to determine the incidence rate of online bugs specifically within CD systems.

2. Reconstructing Natural Language Query with Knowledge Document: Enhancing the query using relevant knowledge documents to provide context. For example, transforming "CD" into "Continuous Delivery" and adding a time range such as "April" to clarify the query.

3. Metric Breakdown and Calculation Logic Planning: Based on the existing sub-metrics and calculation operators, plan the calculation logic. For instance, replace entities with "the number of online bugs" and "total number of bugs," and incorporate computational logic to derive the incidence rate.

4. Planning Query Steps: Output the *Planning Steps* in structured natural language to ensure that the steps are clear and comprehensive.

At this point, the Hierarchical Metric Chain of Thought concludes its output. In the RBDQ system, the subsequent steps involve mapping sub-metrics to SQL templates, identifying relevant tables, columns, and cells, and finally executing the SQL queries or performing operator calculations to generate the final query result.

3.2 RBDQ System

The overall workflow of the RBDQ System is illustrated in Figure 3, showcasing the process from a user's concise input to the system's comprehensive response. The system integrates Retrieval-Augmented Generation (RAG), the Hierarchical Metric Query Method, and a self-reflection mechanism.

In the RAG component, during the pre-retrieval enhancement stage, the user's input is combined with the chat history. This combination helps rewrite and expand the query into a more complete and coherent question. In the retrieval stage, this complete query is used for retrieval, which is divided into two parts: sub-metric retriever and knowledge retriever.

For the sub-metric retriever, we employ the Gpt-ner [24] method to extract all possible metrics. Each extracted metric is then retrieved using the cosine similarity algorithm to obtain sub-metric information.

For the knowledge retriever, we use the MMR algorithm to retrieve relevant knowledge that is not related to sub-metrics, such as department information and metric explanations. The MMR algorithm balances the relevance and diversity of the recommended results. The retrieved knowledge is then passed to the Hierarchical Metric Query Method.

During the Hierarchical Metric Planning stage, the Hierarchical Metric Chain of Thought collaborates with the user's query and RAG documents to break down the query into *Planning Steps*. If these steps involve sub-metric calculations, the Query Sub-metric process is initiated. This process involves extracting the parameters of the planning steps and mapping them to the corresponding sub-metric SQL templates. If the extracted parameters do not match

the required parameters in the template, they are re-extracted and validated to ensure alignment, preventing critical information loss from model hallucinations. Finally, the constructed sub-metric SQL is executed in the database.

In the Generate Answer stage, all computation operators for calculations are implemented as tools accessible to the LLM [31]. The LLM utilizes the content of the *Planning Steps* to compute the retrieved sub-metrics using these computation operators, thereby generating the query result. Subsequently, the self-reflection mechanism is employed to validate both the steps and the computed result, ensuring that the query requirements are met. The self-reflection mechanism verifies the steps and results generated by the LLM. It compares the computed query results against document knowledge, checking for consistency and correctness. If discrepancies are found, the process is revisited, and the steps are refined to ensure accurate results. If the requirements are not satisfied, the Hierarchical Metric Planning process is repeated; if they are satisfied, the result is produced.

For prompt construction, we use the OpenAI Demonstration Prompt, which consists of instructions, table schemas, questions, and context, with all information commented by the pound sign "#". We employ DAIL selection for 5-shot example selection and organization [5].

The output includes the *Planning Steps* taken to generate the result. Since all steps are executed through SQL templates and implemented logical operators, the reliability of the result is ensured, addressing issues of inconsistent SQL output or calculation errors caused by LLM hallucinations and randomness.

4 Experiments

4.1 Experiments Setup

In our experimental setup, we used GPT-4 as the LLM model. By default, the maximum context length was set to 4096 tokens, and the temperature parameter was set to 0, which ensures deterministic output and prevents the model from generating random or diverse responses, improving consistency in the results. Within the RAG system, the Sub-metric Retriever employed a cosine similarity algorithm with a similarity threshold of 0.9, while the Knowledge Retriever used the MMR algorithm to obtain 4 relevant documents. The self-reflection mechanism was configured to allow a maximum of 3 reflection iterations.

To rigorously evaluate the effectiveness of the RBDQ system, we conducted a series of ablation experiments. These experiments were designed to isolate and assess the contributions of individual components within the system. Specifically, we compared the performance of the full RBDQ system against the following three variants:

- (1) **Only Standard RAG Retrieval (Baseline):** This variant uses only cosine similarity for standard RAG retrieval of the most relevant documents, without incorporating the self-reflection mechanism and the Hierarchical Metric Query Method.
- (2) **RBDQ without Self-Reflection (RBDQ-NSR):** This variant evaluates the system's performance without the self-reflection mechanism.

- (3) **RBDQ without Self-Reflection and Hierarchical Metric Query Method (RBDQ-NSR-NHM):** This variant assesses the system's performance when both the self-reflection mechanism and the Hierarchical Metric Query Method are removed.

These comparisons enabled us to systematically evaluate the impact of each component on the overall system performance, providing insights into the contributions of the self-reflection mechanism and the Hierarchical Metric Query Method.

4.2 Datasets

Due to the lack of publicly available datasets that align with our task's specific requirements, such as comprehensive business knowledge and auxiliary information, we created a custom dataset from the Quality Assurance department at ByteDance. This dataset includes 108 business-related query phrases and leverages the ByteDance Quality Assurance department's knowledge base for document retrieval. Examples of the dataset are shown in Table 1.

We extracted 46 sub-metrics, with examples shown in Table 2.

4.3 Metrics

We used Execution Accuracy (EX) [27] to evaluate our system. This metric evaluates the performance of the Text-to-SQL system by comparing whether the execution result sets of the ground-truth SQL queries and the predicted SQL queries are identical. It can be computed as:

$$EX = \frac{\sum_{i=1}^N \mathbb{I}(V_i = \hat{V}_i)}{N}$$

where $\mathbb{I}(\cdot)$ is an indicator function that equals 1 if the condition inside is satisfied, and 0 otherwise. Note that false negatives could occur because different SQL queries corresponding to semantically different NL queries may produce identical execution result sets, which means that while the queries may be different, they could still return the same outcome.

4.4 Results

The results of our experiments are summarized in Table 3. The EX for each variant is as follows:

The baseline system, which uses only standard RAG retrieval, achieved an Execution Accuracy of 72.22%. This relatively low performance underscores the limitations of relying solely on cosine similarity for document retrieval without additional mechanisms to enhance query understanding and accuracy.

The RBDQ-NSR-NHM variant, which excludes both the self-reflection mechanism and the Hierarchical Metric Query Method, showed a significant improvement with an Execution Accuracy of 79.62%. This indicates that even with knowledge base augmentation and chain-of-thought techniques, achieving business-level usability remains challenging. This result highlights the limitations of relying solely on these methods.

The RBDQ-NSR variant, which includes the Hierarchical Metric Query Method but not the self-reflection mechanism, achieved an Execution Accuracy of 92.59%. This substantial increase demonstrates the critical role of the Hierarchical Metric Query Method in

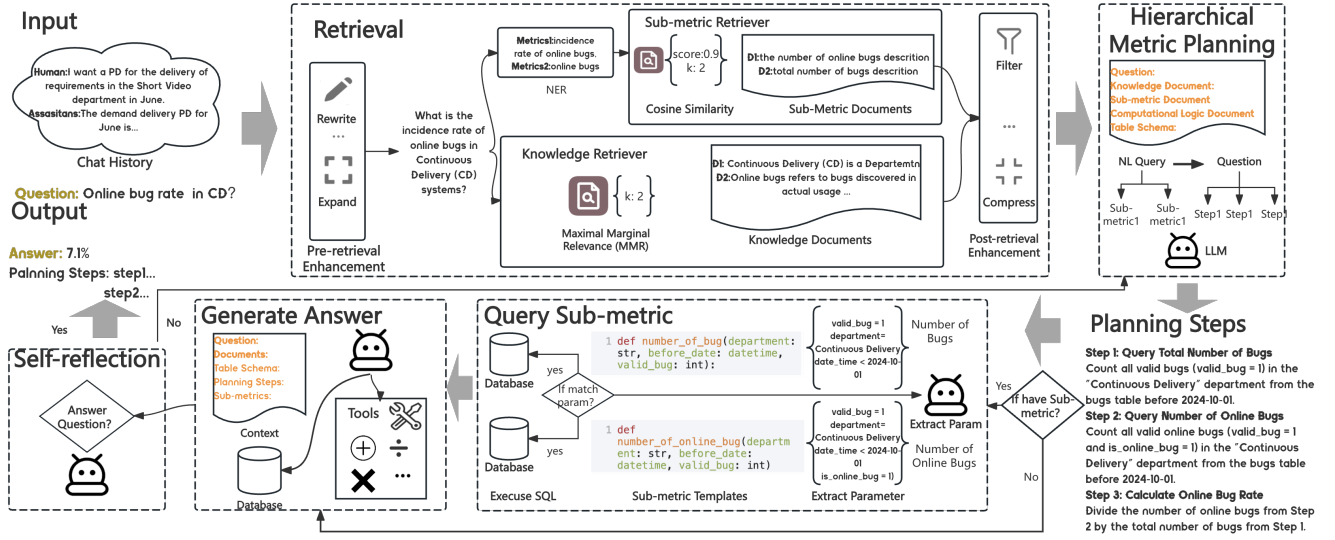


Figure 3: RBDQ System

enhancing the system's ability to accurately interpret and execute complex queries.

The full RBDQ system, which incorporates both the self-reflection mechanism and the Hierarchical Metric Query Method, achieved the highest Execution Accuracy of 96.20%. This result underscores the importance of the self-reflection mechanism in further refining the system's outputs and ensuring high reliability and accuracy in query execution. The improvement is particularly noticeable in complex queries, where the LLM, based on its pre-trained knowledge, identifies inadequacies in the initial *Planning Steps* and iteratively refines them to meet the query requirements, thereby enhancing accuracy.

5 Discussion

5.1 Why is RBDQ Reliable?

Reliability is a critical concern in business queries. LLMs are expected to use chain-of-thought reasoning to output query steps. However, LLMs may probabilistically generate SQL that either deviates or ignores conditions from the planned query steps, leading to user distrust in text-to-SQL systems.

RBDQ achieves reliability by mapping query statements to corresponding metric SQL templates, ensuring that the generated SQL aligns with the planned query steps. Additionally, the calculation logic is implemented through pre-defined computation operators in code, rather than relying on the LLM to generate executable code. This approach guarantees 100% accuracy in the translation of query steps to result, a level of precision that even the most advanced LLMs (e.g., GPT-o1) cannot consistently achieve.

Even if errors occur in the *Planning Steps*, business users can easily trace the issues to determine the usability of the output results, thereby making the system suitable for real-world applications.

Consequently, users can place their trust in the results produced by RBDQ.

5.2 The effectiveness of RBDQ in Real-World Scenarios

RBDQ has been deployed in the Quality Assurance department of ByteDance, resulting in approximately a 5% increase in operational efficiency for the quality assurance team. The system has enhanced the flexibility of queries and reduced the learning curve for users, while also contributing to the enrichment of the business team's knowledge base.

We analyzed 4 instances where queries failed. One failure was due to the complexity of the query, which involved 4 sub-queries and 2 calculation logics. The other 3 failures were due to the absence of definitions in the knowledge document, such as the term "production capacity" in the query "Query the top 5 people with the highest production capacity in the QA team." These issues can be resolved by supplementing the knowledge base with definitions for complex metrics or specific terms. Therefore, while RBDQ may occasionally struggle with overly complex query logic, targeted enhancements to the knowledge base could potentially achieve 100% execution accuracy.

5.3 Limitations and Future Work

RBDQ necessitates extensive extraction of sub-metrics and refinement of computation operators. Although the approach to sub-metric extraction is capable of handling complex queries, it encounters challenges with highly intricate planning problems that involve more than five query steps. Furthermore, more computation operators need to be implemented in code, which could be improved by incorporating database operators in future iterations.

Table 1: Example Data

Query	Tell me the percentage of 0Bug personnel for Lemon8 last month, distinguishing between full-time and outsourced
Planning Step	Query the number of 0Bug personnel: - Time: p_date (YYYY-MM) = 'October 2024' - Department: 'Lemon8' - Other conditions: whitelist = 0, valid_bug = 0 - Employee types: 'Regular' and 'Outsourced' - Aggregate by employee_type, count the number of 0Bug personnel Query the total number of personnel: - Time: p_date (YYYY-MM) = 'October 2024' - Department: 'Lemon8' - Other conditions: whitelist = 0 - Employee types: 'Regular' and 'Outsourced' - Aggregate by employee_type, count the total number of personnel Calculate the proportion of 0Bug personnel: - Calculate the ratio of 0Bug personnel to the total number of personnel, list employee_type, and percentage of 0Bug personnel respectively
Result	employee_type,0Bug Personnel Percentage Outsourced,0.032 Full-time,0.032
Query	Percentage of effort spent on Client Arch regression in September
Planning Step	Query total working hours: - Time: start_time <= 'September 30, 2024', finish_time >= 'September 1, 2024' - Department: 'Client Arch' - Other conditions: within_statistics_scope = 1 - List fields: department, working hours, and sum (weekly dimension) as total working hours Query weekly working hours: - Time: start_time <= 'September 30, 2024', finish_time >= 'September 1, 2024' - Department: 'Client Arch' - Other conditions: within_statistics_scope = 1, Task distribution_aggregation is 'Requirement Delivery - Regression' - List fields: department, working hours, and sum (weekly dimension) as weekly working hours Calculate the percentage of effort spent on regression: - Divide the weekly working hours of each Value_Chain by the total working hours of each Value_Chain List fields: percentage of effort spent on regression
Result	0.11812

To address these limitations, future work on RBDQ could focus on integrating more comprehensive database operators to broaden the range of supported calculations. Additionally, the introduction of a multi-agent mechanism could facilitate the handling of more complex query logic and enable a wider array of analytical functions beyond data querying, such as generating detailed analytical reports.

6 Conclusion

In this paper, we introduced RBDQ, an LLM-based text-to-SQL system tailored for business data queries. Our approach combines advanced RAG methods with a self-reflection mechanism to enhance problem-solving performance. And We proposed a novel hierarchical metrics query method to balance robustness and performance, ensuring reliable and accurate query results. Ablation experiments demonstrated the effectiveness of each RBDQ component, with the

Table 2: Example of Sub-metric

Metric	Description	Aliases	SQL Query
Total Valid Bugs Found by QA	The total number of valid bugs found by QA.	total_valid_bugs, valid_bug_sum, QA_valid_bugs	SELECT SUM(valid_bug) AS total_valid_bugs FROM performance WHERE department != 'C';
High Priority Online Bugs	The number of high priority online bugs.	critical_online_bugs, P0P1_bugs	SELECT COUNT(*) AS high_priority_online_bugs FROM bugs WHERE discovery_phase_CN IS NOT NULL AND is_test_bug = 0 AND online_user_unaware_bug = 0 AND is_valid_bug = 1 AND bugbash_bug = 0;

Table 3: Execution Accuracy (EX) of Different Variants

Variant	EX (%)
Baseline (Only Standard RAG Retrieval)	72.22
RBDQ-NSR-NHM	79.62
RBDQ-NSR	92.59
RBDQ (Full System)	96.20

Hierarchical Metric Query Method improving the system’s ability to handle complex queries and the self-reflection mechanism refining outputs for high accuracy. The full RBDQ system achieved an Execution Accuracy of 96.20%, outperforming variants without these advanced mechanisms.

RBDQ has been successfully deployed in ByteDance’s Quality Assurance department, increasing operational efficiency and improving query process flexibility and usability. This deployment highlights RBDQ’s practical value in business environments. However, RBDQ has limitations, such as the need for extensive sub-metric extraction and calculation logic refinement. Future work will focus on integrating more comprehensive database operators and introducing a multi-agent mechanism to handle more complex query logic. In conclusion, RBDQ represents a significant advancement in text-to-SQL systems, offering a robust, accurate, and reliable solution for business data queries.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grants 62137001, 62277017, 61977026, and 62072115. Additionally, this work was supported by the Shanghai Science and Technology Innovation Action Plan Project under Grant No. 22510713600.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [4] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. 2021. Structure-Grounded Pretraining for Text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 1337–1350. <https://doi.org/10.18653/v1/2021.naacl-main.105>
- [5] Dawei Gao, Haibin Wang, Yaliang Li, Xiyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (Jan. 2024), 1132–1145. <https://doi.org/10.14778/3641204.3641221>
- [6] Linda Hammami, Alessia Paglialonga, Giancarlo Pruneri, Michele Torresani, Milena Sant, Carlo Bono, Enrico Gianluca Caiani, and Paolo Baili. 2021. Automated Classification of Cancer Morphology from Italian Pathology Reports Using Natural Language Processing Techniques: A Rule-Based Approach. *Journal of Biomedical Informatics* 116 (2021), 103712.
- [7] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2024. Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL. *arXiv preprint arXiv:2406.08426* (2024).
- [8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2024. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* (2024).
- [9] Ning Kang, Bharat Singh, Zubair Afzal, Erik M van Mulligen, and Jan A Kors. 2013. Using Rule-Based Natural Language Processing to Improve Disease Normalization in Biomedical Text. *Journal of the American Medical Informatics Association* 20, 5 (2013), 876–881.
- [10] George Katsogiannis-Meimarakis and Georgia Koutrika. 2021. A Deep Dive into Deep Learning Approaches for Text-to-SQL Systems. In *Proceedings of the 2021 International Conference on Management of Data*. 2846–2851.
- [11] George Katsogiannis-Meimarakis and Georgia Koutrika. 2023. A Survey on Deep Learning Approaches for Text-to-SQL. *The VLDB Journal* 32, 4 (2023), 905–936.
- [12] Brenden Lake and Marco Baroni. 2018. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 2873–2882.
- [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [14] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. *Advances in Neural Information Processing Systems* 36 (2024).
- [15] Mohammadreza Pourreza and Davood Rafiei. 2024. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *Advances in Neural Information Processing Systems* 36 (2024).
- [16] Alec Radford and Karthik Narasimhan. 2018. Improving Language Understanding by Generative Pre-Training. <https://api.semanticscholar.org/CorpusID:49313245>
- [17] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, Jian Su, Kevin Duh, and Xavier Carreras (Eds.). Association for Computational

- Linguistics, Austin, Texas, 2383–2392. <https://doi.org/10.18653/v1/D16-1264>
- [18] Matthew Renze and Erhan Guven. 2024. Self-Reflection in LLM Agents: Effects on Problem-Solving Performance. *arXiv preprint arXiv:2405.06682* (2024).
- [19] Zhao Tan, Xiping Liu, Qing Shu, Xi Li, Changxuan Wan, Dexi Liu, Qizhi Wan, and Guoqiong Liao. 2024. Enhancing Text-to-SQL Capabilities of Large Language Models through Tailored Promptings. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (Eds.). ELRA and ICCL, Torino, Italia, 6091–6109. <https://aclanthology.org/2024.lrec-main.539/>
- [20] Dayton G Thorpe, Andrew J Duberstein, and Ian A Kinsey. 2024. Dubo-SQL: Diverse Retrieval-Augmented Generation and Fine Tuning for Text-to-SQL. *arXiv preprint arXiv:2404.12560* (2024).
- [21] Immanuel Trummer. 2022. CodexDB: Synthesizing code for query processing from natural language instructions using GPT-3 Codex. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2921–2928.
- [22] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, LinZheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. 2025. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 540–557. <https://aclanthology.org/2025.coling-main.36/>
- [23] Haifeng Wang, Jiwei Li, Hua Wu, Eduard Hovy, and Yu Sun. 2023. Pre-Trained Language Models and Their Applications. *Engineering* 25 (2023), 51–65.
- [24] Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, and Guoyin Wang. 2023. Gpt-ner: Named entity recognition via large language models. *arXiv preprint arXiv:2304.10428* (2023).
- [25] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
- [26] Haoran Yang, Yumeng Zhang, Jiaqi Xu, Hongyuan Lu, Pheng-Ann Heng, and Wai Lam. 2024. Unveiling the Generalization Power of Fine-Tuned Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 884–899. <https://doi.org/10.18653/v1/2024.naacl-long.51>
- [27] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 3911–3921. <https://doi.org/10.18653/v1/D18-1425>
- [28] Bin Zhang, Yuxiao Ye, Guoqing Du, Xiaoru Hu, Zhishuai Li, Sun Yang, Chi Harold Liu, Rui Zhao, Ziyue Li, and Hangyu Mao. 2024. Benchmarking the Text-to-SQL Capability of Large Language Models: A Comprehensive Evaluation. *arXiv preprint arXiv:2403.02951* (2024).
- [29] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning. *arXiv preprint arXiv:1709.00103* (2017).
- [30] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625* (2022).
- [31] Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2024. Toolqa: A dataset for llm question answering with external tools. *Advances in Neural Information Processing Systems* 36 (2024).